Summary in Graph

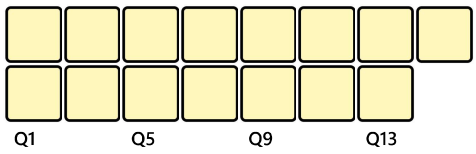
Exam Summary (GO Classes DA Test Series 2026 | Data Structures and Algorithms | Topic Wise Test 1)

Qs. Attempted:	0 (0 + 0)	Correct Marks:	0 (0 + 0)
Correct Attempts:	0 (0 + 0)	Penalty Marks:	0 (0 + 0)
Incorrect Attempts:	0 (0 + 0)	Resultant Marks:	0 (0 + 0)

Total Questions:	15 (5 + 10)
Total Marks:	25 (5 + 20)
Exam Duration:	45 Minutes
Time Taken:	0 Minutes

Individual Question Statistics

Select Metric: Marks Obtained



* Average Stats based on first-attempt stats of Top 10% (58) out of 581 users

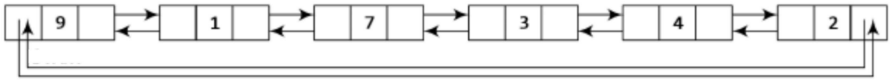
EXAM RESPONSE | EXAM STATS | FEEDBACK

Technical

Q #1 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Data Structures

Consider a function findCLL that takes a doubly circular linked list head and an integer value as input.

One example of doubly circular linked list is given below –




```
    def findCLL(first, n):
        while first.value != n:
            first = first.next

5.      if first.value == n:
            return 1
        else:
            return -1
```

Consider two statements $S1$ and $S2$ given below.

- $S1$: Function returns 1 if there exists a value in linked list
- $S2$: Function returns -1 if value does not exist in linked list

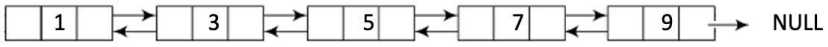
Which of the following is the correct option.

A. $S1$ is True but $S2$ is False.
B. $S2$ is True but $S1$ is False.
C. Both are True.
D. Both are False.

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 03sec | Discuss

Q #2 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Data Structures

Consider the following Doubly Linked List:



If head points to the first node of the linked list then what will be the output of the following node?

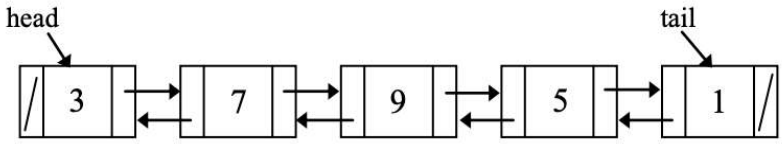
```
    head = head.next.next.next.prev
    head.next.next.prev = head
    print(head.next.next.prev.next.value)
```

A. 1
B. 3
C. 5
D. 7

Your Answer: | Correct Answer: D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #3 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Data Structures

Given a doubly linked list where each node has two references (prev and next): one that points to a previous node and another that points to a next node. Assume the linked list below and provide the output for the following two lines.



The list is restored to its initial state before each line executes:

```
Line1: print(head.next.next.next.prev.prev.data)
Line2: print(tail.prev.prev.prev.prev.next.next.data)
```

Which of the options is correct.

A. Output of line 1 is 7 and Output of line 2 is 9
B. Output of line 1 is 9 and Output of line 2 is 7
C. Both lines output 7
D. Both lines output 9

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #4 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Data Structures

What is the run-time complexity of inserting a new element at the beginning of a circular, doubly linked list with a head?

A. $O(1)$

B. $O(\log \mathrm{N})$

C. $O(\mathrm{N})$

D. $O(\mathrm{N}^2)$

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #5 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Data Structures

Given a circular, doubly-linked list whose contents are sorted in ascending order, what is the run-time complexity for inserting a new element into the list so that it remains correctly sorted? (Including the time required to search for the element's correct position.)

A. $O(1)$

B. $O(\log \mathrm{N})$

C. $O(N)$

D. $O(\mathrm{N}^2)$

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #6 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

```
    class Node:
        def __init__(self, data, next=None):
            self.data = data
            self.next = next
 5.
    def print_nodes(ptr):
        if ptr:
            print(ptr.data, end=' ')
            while ptr.next:
10.             ptr = ptr.next
                print(ptr.data, end=' ')
```

What is the output, if the address of the first node of singly linked list $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ is passed in the above python code?

A. $1\ 2\ 3\ 4\ 5$
B. $1\ 1\ 2\ 3\ 4\ 5$
C. $1\ 1\ 2\ 3\ 4\ 5\ 5$
D. None of these

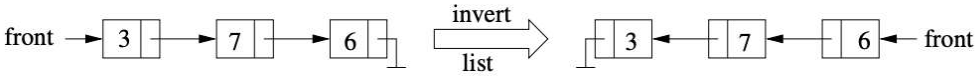
Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #7 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Consider the following code fragment.

```
    class Node:
        def __init__(self, data, next=None):
            self.data = data
            self.next = next
 5.
    def invert_list(front):
        curr = front
        prev = None
        next = curr.next
10.     while curr is not None :
            (*)
        front = prev
        return front
```

Which code must be added in the part marked (*) so the above code correctly inverts a non-empty singly linked list? See the figure to understand what "invert" means.



A.
```
    next.next = prev; prev = curr; curr = next;
    if next is not None: next = next.next
```

B.
```
    curr.next = prev; prev = curr; curr = next_node;
    if next is not None: next = next.next
```

C.
```
    next.next = curr; prev = curr; curr = next;
    if next is not None: next = next.next
```

D.
```
    prev = curr; curr = next; curr.next = prev;
    if next is not None: next = next.next
```

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #8 | Multiple Select Type | Award: 2 | Penalty: 0 | Data Structures

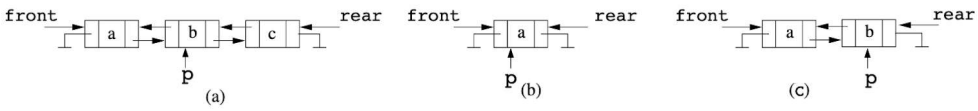
The following code is intended to remove a node p from a doubly linked list. Assume that we know that p is in the list, so the list is not empty.

```
    class Node:
        def __init__(self, value, next=None, prev=None):
            self.value = value
            self.next = next
 5.         self.prev = prev

    def remove_node(p, front, rear):
        prev = p.prev
        succ = p.next
10.
        if p == front:
            front = front.next
            if front is None:
                rear = None
15.         else:
                front.prev = None
        else:
            prev.next = succ

20.     if p == rear:
            rear = rear.next
            rear.next = None
        else:
            succ.prev = prev
25.
        return front, rear
```

Given three linked lists (a),(b), and (c) as shown below.



Mark the correct option(s).

A. Code will crash on linked list (a)
B. Code will crash on linked list (b)
C. Code will crash on linked list (c)
D. Code will not crash on any of the given linked lists

Your Answer: | Correct Answer: B;C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #9 | Multiple Select Type | Award: 2 | Penalty: 0 | Data Structures

Consider a mutual pair of recursive functions g() and h().

```
    class Node:
        def __init__(self, value, next=None):
            self.value = value
            self.next = next
 5.
    def g(l):
        if l is None or l.next is None:
            return 1
        if l.value < l.next.value:
10.         return h(l.next)
        else:
            return 0

    def h(l):
15.     if l is None or l.next is None:
            return 1
        if l.value > l.next.value:
            return g(l.next)
        else:
20.         return 0
```

Let head be a pointer to a singly linked list having at least 3 nodes.

When will the expression (g(head) || h(head)) return 1 or 0?

A. g(head) || h(head) is 1 if the linked list is in ascending order.
B. g(head) || h(head) is 1 if the linked list is in descending order.
C. g(head) || h(head) is 1 for every unsorted linked list.
D. g(head) || h(head) is 1 for the linked list $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 0 \rightarrow 6$

Your Answer: | Correct Answer: D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #10 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Consider the following function that takes reference to head of a Doubly Linked List as parameter. Assume that a node of doubly linked list has previous pointer as $prev$ and next pointer as $next$.

```
    class Node:
        def __init__(self, value):
            self.value = value
            self.prev = None
 5.         self.next = None

    def fun(head_ref):
        temp = None
        current = head_ref
10.     while current is not None:
            temp = current.prev
            current.prev = current.next
            current.next = temp
            current = current.prev
15.     if temp is not None:
            head_ref = temp.prev
```

Assume that reference of head of following doubly linked list is passed to above function

$1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5 \leftrightarrow 6$. What should be the modified linked list after the function call?

A. $2 \leftrightarrow 1 \leftrightarrow 4 \leftrightarrow 3 \leftrightarrow 6 \leftrightarrow 5$
B. $5 \leftrightarrow 4 \leftrightarrow 3 \leftrightarrow 2 \leftrightarrow 1 \leftrightarrow 6$
C. $6 \leftrightarrow 5 \leftrightarrow 4 \leftrightarrow 3 \leftrightarrow 2 \leftrightarrow 1$
D. $6 \leftrightarrow 5 \leftrightarrow 4 \leftrightarrow 3 \leftrightarrow 1 \leftrightarrow 2$

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #11 | Multiple Select Type | Award: 2 | Penalty: 0 | Data Structures

Consider the following function Merge() that takes the head of two linked lists.

```
    class Node:
        def __init__(self, value):
            self.value = value
            self.next = None
 5.
    def Merge(head1, head2):
        if head1 is None:
            return head2
        if head2 is None:
10.         return head1
        head = None
        if head1.value < head2.value:
            head = head1
            head.next = Merge(head1.next, head2)
15.     elif head1.value > head2.value:
            head = head2
            head.next = Merge(head1, head2.next)
        return head
```

Assume that the input lists are correctly sorted. Which of the following are some of the possible behaviors when Merge() is executed with well-formed and valid inputs? Correctly merged linked list is merged sorted linked list.

A. The function will produce a correctly merged linked list.
B. The function may lead to a null pointer dereference.
C. The function may result in an incorrectly merged linked list.
D. Merge() will work on the following two lists.
   - $\text{List1} : 1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow \text{Null}$
   - $\text{List2} : 2 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 10 \rightarrow 12 \rightarrow \text{Null}$

Your Answer: | Correct Answer: C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #12 | Multiple Select Type | Award: 2 | Penalty: 0 | Data Structures

Consider the following program. printlist() is a function that takes the head of a linked list and prints all nodes values separated by comma. Node can be assumed to be declared as class Node.

```
    def insert1(head, data):
        new_node = Node(data)
        new_node.next = head
        head = new_node
 5.
    def insert2(head_ref, data):
        new_node = Node(data)
        new_node.next = head_ref[0]
        head_ref[0] = new_node
10.
    # create a linked list 1->2->3->4->5 and head points to the first node.
    head = insert1(head, 9)
    printlist(head)  # Line X

15. # The list is restored to its initial state
    head_ref = [head]
    insert2(head_ref, 9)
    printlist(head_ref[0])  # Line Y
```

Which of the following is/are true about the above program?

A. Line X prints $9, 1, 2, 3, 4, 5$
B. Line Y prints $9, 1, 2, 3, 4, 5$
C. Line X prints $1, 2, 3, 4, 5$
D. Line Y prints $1, 2, 3, 4, 5$

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #13 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Consider the following function fun() that takes the head of a linked list.

```
    class Node:
        def __init__(self, value=0, next=None):
            self.value = value
            self.next = next
 5.
    def fun(head):
        if head is None:
            return 1
        p = head
10.     q = p.next
        while q is not None and q != p:
            q = q.next
            if q is None:
                return 1
15.         q = q.next
            p = p.next
        return 1 if q is None else 0
```

We say, a linked list has a loop if the last node of linked list points to some node of linked list and does not point to NULL.

What does the above function do?

A. Returns 0 is there is loop in linked list
B. Returns 1 is there is loop in linked list
C. Returns 0 is length of the linked list is even
D. Function may go to infinite loop if there is a loop in linked list

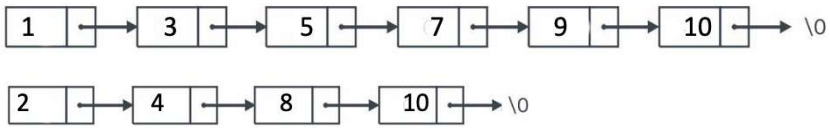
Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #14 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Consider the following function foo() which takes the head pointer of two singly-linked lists.

```
    class Node:
        def __init__(self, value=0, next=None):
            self.value = value
            self.next = next
 5.
    def foo(head1, head2):
        if head1 is None:
            return head2
        if head2 is None:
10.         return head1
        temp = foo(head1.next, head2.next)
        final = head1
        head1.next = head2
        head2.next = temp
15.     return final
```

What will be the final linked list returned by foo() if executed upon following linked lists?



A. $1, 2, 3, 4, 5, 7, 8, 9, 10$
B. $1, 2, 3, 4, 5, 7, 8, 10, 9, 10$
C. $1, 2, 3, 4, 5, 8, 7, 10, 9, 10$
D. None of these

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #15 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

A doubly linked list is declared as:

```
    class Node:
        def __init__(self, value=0, prev=None, next=None):
            self.value = value
            self.prev = prev
 5.         self.next = next
```

Which of the following segment of code deletes the node pointed to by X from the doubly linked list, if it is assumed that X points to neither the first nor the last node of the list?

A. X.prev.next $=$ X.next ; X.next.prev $=$ X.prev; free(X);
B. X.prev.next $=$ X.next ; X. next.prev $=$ X.prev; free(X);
C. X.prev.next $=$ X.prev ; X.next.prev $=$ X.prev; free(X);
D. X.prev.next $=$ X.prev ; X.next.prev $=$ X.next; free(X);

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Copyright & Stuff